

Programming and Algorithms

Aula 14: Matrizes e Tabelas

Cópias de objetos

Prof. Ettore Baldini-Neto

ettore.neto@faculdadeimpacta.com.br

Programa da Aula

- Introdução às sequências aninhadas.
- Sequências aninhadas para construir matrizes e tabelas
- Objetos mutáveis e imutáveis: parte 2
- Como ocorre a cópia de objetos em Python
- Cópias rasas x cópias profundas
- Como chamar funções com argumentos nomeados e como defini-las com argumentos padrão.

Introdução

- Vimos nas últimas aulas que sequências são úteis para tratar de uma grande quantidade de dados.
  - **Exemplos:** Expectativa de vida de uma dada população, altura média da mesma, folhas de pagamentos de grandes empresas, contagem de itens em estoques, etc.
- Nestes exemplos laços de repetição podem ser utilizados para iterar sobre as sequências e realizar operações que seriam impossíveis se dependessemos somente de variáveis simples.

Motivações

Existem problemas cujas informações **estão aninhadas** como, por exemplo, guardar as notas de todos os alunos de todas as turmas de uma instituição de ensino.

Temos aqui: Lista de turmas, nas quais cada turma é uma lista de alunos e para cada aluno temos uma lista de notas.

Ou seja, temos sequências de sequências ou sequências aninhadas. Vamos tratar disso.

Sequências Aninhadas

- Listas e tuplas são sequências que podem conter itens de qualquer tipo, inclusive outras sequências.
- Para que uma sequência seja aninhada basta coloca-la **como item de outra sequência.**

Exemplo 1: Aninhamento com o objetos literais

```
>>> s = [('pt', 'arroz'), ('en', 'rice'), ('fr', 'riz'), ('de', 'reis')]
>>> s
[('pt', 'arroz'), ('en', 'rice'), ('fr', 'riz'), ('de', 'reis')]
>>>
```

Sequências Aninhadas

Exemplo 2: Aninhamento com objetos que sejam referenciados por variáveis

```
>>> t_1 = [10, 11, 12]
>>> t_2 = [20, 21, 22]
>>> t_3 = [30, 31, 32]
>>> t = [t_1, t_2, t_3]
>>> t
[[10, 11, 12], [20, 21, 22], [30, 31, 32]]
>>>
```

Para acessar itens de uma sequência aninhada, **basta encadear índices.**

Sequências Aninhadas

```
>>> s = [('pt', 'arroz'), ('en', 'rice'), ('fr', 'riz'), ('de', 'reis')]
>>> s
[('pt', 'arroz'), ('en', 'rice'), ('fr', 'riz'), ('de', 'reis')]
>>>

>>> s[0]
('pt', 'arroz')
>>> s[1]
('en', 'rice')
>>> s[1][1]
'rice'
>>> s[1][0]
'en'
>>> s[2][1]
'riz'
>>> s[3][3]
Traceback (most recent call last):
  File "<pyshell#20>", line 1, in <module>
    s[3][3]
IndexError: tuple index out of range
>>>
```

Sequências Aninhadas

```
>>> t_1 = [10, 11, 12]
>>> t_2 = [20, 21, 22]
>>> t_3 = [30, 31, 32]
>>> t = [t_1, t_2, t_3]
>>> t
[[10, 11, 12], [20, 21, 22], [30, 31, 32]]
>>>
```

```
>>> t[0][3]
Traceback (most recent call last):
  File "<pyshell#22>", line 1, in <module>
    t[0][3]
IndexError: list index out of range
>>> t[0]
[10, 11, 12]
>>> t[0][2]
12
>>> t[1][1]
21
>>> t[2][0]
30
>>>
```

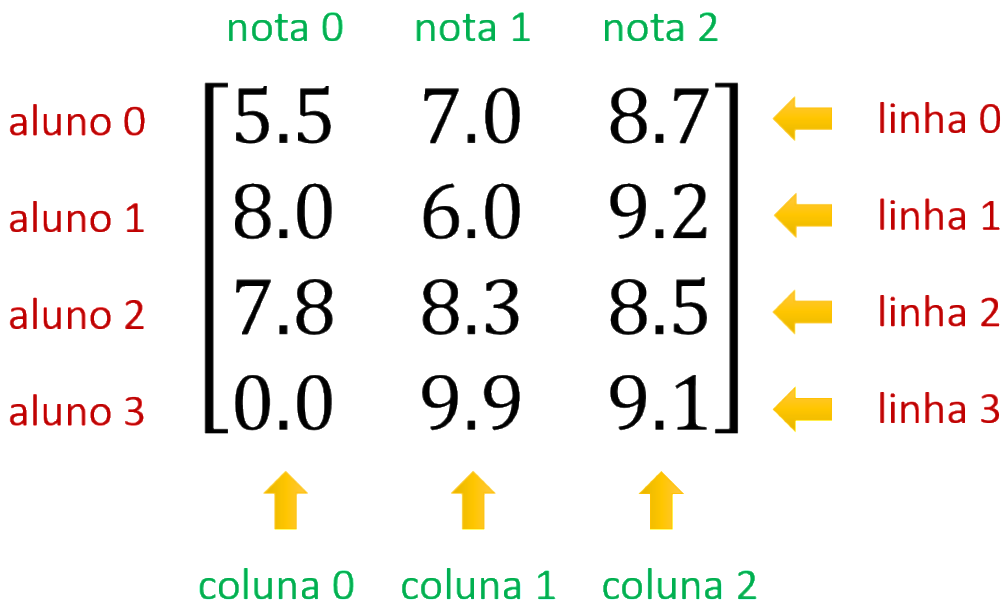
Matrizes e Tabelas

Uma utilização natural das sequências aninhadas é a representação de matrizes e tabelas.

- **Matrizes** são formadas por elementos que - no âmbito geral podem ter qualquer natureza- normalmente são números ou resultados de operações numéricas dispostos em linhas e colunas
- **Tabelas** são semelhantes às matrizes mas normalmente seus itens são mais diversos e podem ser de tipos distintos. Suas linhas são chamadas registros e cada coluna da linha é um campo daquele registro.

Matrizes e Tabelas

Exemplo 3: Matriz de notas



Matriz numérica

4x3

(4 linhas por 3 colunas)

Vejamos dois modos de escrever esta matriz em Python

Matrizes e Tabelas

1. Com variáveis simples: 12 variáveis simples (atribuição paralela)

```
aluno_0_0, aluno_0_1, aluno_0_2 = 5.5, 7.0, 8.7
aluno_1_0, aluno_1_1, aluno_1_2 = 8.0, 6.0, 9.2
aluno_2_0, aluno_2_1, aluno_2_2 = 7.8, 8.3, 8.5
aluno_3_0, aluno_3_1, aluno_3_2 = 0.0, 9.9, 9.1
```

Codificação 12.4: Representação da matriz da Figura 12.1 com variáveis simples.

2. Com uma lista para cada aluno

```
aluno_0 = [5.5, 7.0, 8.7]
aluno_1 = [8.0, 6.0, 9.2]
aluno_2 = [7.8, 8.3, 8.5]
aluno_3 = [0.0, 9.9, 9.1]
```

Codificação 12.5: Representação da matriz da Figura 12.1 com listas simples.

Matrizes e Tabelas

Vamos escrever uma função para calcular a media dos alunos com quatro chamadas da função calcula média

```
def calcula_media(aluno):
    soma = 0
    for nota in aluno:
        soma += nota
    return soma/len(aluno)
```

Calcule a média para os alunos abaixo.

```
aluno_0 = [5.5, 7.0, 8.7]
aluno_1 = [8.0, 6.0, 9.2]
aluno_2 = [7.8, 8.3, 8.5]
aluno_3 = [0.0, 9.9, 9.1]
```

Codificação 12.5: Representação da matriz da Figura 12.1 com listas simples.

Matrizes e Tabelas

- Vamos representar a mesma matriz de uma maneira mais "esperta" utilizando aninhamento de sequência

```
alunos = [[5.5, 7.0, 8.7],
          [8.0, 6.0, 9.2],
          [7.8, 8.3, 8.5],
          [0.0, 9.9, 9.1]]
```

Cada linha representa um aluno e cada coluna representa as notas desse aluno.

Matrizes e Tabelas

Calculando a média

Um laço para percorrer as linhas (alunos) e para cada linha usar outro laço para percorrer suas colunas (notas)

```
alunos = [[5.5, 7.0, 8.7],
          [8.0, 6.0, 9.2],
          [7.8, 8.3, 8.5],
          [0.0, 9.9, 9.1]]

medias = []
for aluno in alunos:
    soma = 0
    for nota in aluno:
        soma += nota
    media = soma / len(aluno)
    medias.append(media)
print(medias)
```

Matrizes e Tabelas

Podemos melhorar a codificação escrevendo também uma função que recebe como argumento uma matriz de notas dos alunos de uma turma e retornará uma lista com as médias destes. Assim, a função pode ser utilizada para o cálculo da média de diversas turmas.

Matrizes e Tabelas

```
def calcula_media(aluno):
    soma = 0
    for nota in aluno:
        soma += nota
    return soma/len(aluno)

def calcula_medias(turma):
    medias = []
    for aluno in turma:
        media = calcula_media(aluno)
        medias.append(media)
    return medias
```

Matrizes e Tabelas

Em Python é comum evitar a alteração dos argumentos recebidos pelas funções, mas isso depende do problema a ser tratado. Vamos tratar o seguinte exemplo para ver que nem sempre é assim.

Exemplo: Crie uma função que receba como argumento uma matriz de notas na qual cada linha representa as notas de um aluno na escala de 0 até 10. Esta função deve atualizar a matriz para a escala de 0 até 20, mantendo a proporção.

Vamos ver três soluções propostas:

Matrizes e Tabelas

1. A função abaixo atualiza as notas, **sem modificar o argumento**, ou seja, **sem gerar efeitos colaterais**

```
def atualiza_nota_1(turma):
    turma_atualizada = []
    for aluno in turma:
        aluno_atualizado = []
        for nota in aluno:
            aluno_atualizado.append(nota * 2)
        turma_atualizada.append(aluno_atualizado)
    return turma_atualizada
```

Matrizes e Tabelas

2.A função atualiza_notas_2 não preserva o argumento. A ideia é justamente alterá-lo. Observe que os itens de turma não precisaram ser alterados, somente os itens da sequência aninhada em turma. Logo o laço externo não precisa ser alterado, **apenas o laço interno, que agora vai operar sobre os índices de cada aluno, para que os valores sejam alterados.**

```
def atualiza_notas_2(turma):
    for aluno in turma:
        for i in range(len(aluno)):
            aluno[i] *= 2
```

Matrizes e Tabelas

3. Ainda podemos escrever uma codificação equivalente em que os itens da matriz são acessados por meio de seus dois índices, ou seja, linha e coluna

```
def atualiza_notas_3(turma):
    for linha in range(len(turma)):
        for coluna in range(len(turma[linha])):
            turma[linha][coluna] *= 2
```

Matrizes e Tabelas

```
def atualiza_nota_1(turma):
    turma_atualizada = []
    for aluno in turma:
        aluno_atualizado = []
        for nota in aluno:
            aluno_atualizado.append(nota * 2)
        turma_atualizada.append(aluno_atualizado)
    return turma_atualizada


def atualiza_notas_2(turma):
    for aluno in turma:
        for i in range(len(aluno)):
            aluno[i] *= 2


def atualiza_notas_3(turma):
    for linha in range(len(turma)):
        for coluna in range(len(turma[linha])):
            turma[linha][coluna] *= 2
```

Matrizes e Tabelas

```
def exibe(turma):
    for i in range(len(turma)):
        for j in range(len(turma[i])):
            print(f'{turma[i][j]:5.2f}', end=' ')
        print()

def atualiza_notas_2(turma):
    for aluno in turma:
        for i in range(len(aluno)):
            aluno[i] *= 2

alunos = [[5.5, 7.0, 8.7],
          [8.0, 6.0, 9.2],
          [7.8, 8.3, 8.5],
          [0.0, 9.9, 9.1]]

print('Antes')
exibe(alunos)

atualiza_notas_2(alunos)

print('\nDepois')
exibe(alunos)
```

Preenchendo Matrizes e Tabelas

No programa mostrado a seguir, a função **coleta_notas** preenche e retorna uma lista de notas de um aluno enquanto que a função **preenche_turma** chama diversas vezes a função **coleta_notas** para preencher a matriz de notas de uma turma com **qtd_alunos** linhas e, ao final, retorna a matriz criada.

Matrizes e Tabelas: Outro exemplo

```
def coleta_notas():
    notas = input().split()
    for i in range(len(notas)):
        notas[i] = float(notas[i])
    return notas

def preenche_turma(qtd_alunos):
    turma = []
    for i in range(qtd_alunos):
        print(f'{i + 1} aluno:', end='')
        aluno = coleta_notas()
        turma.append(aluno)
    return turma

def calcula_media(aluno):
    soma = 0
    for nota in aluno:
        soma += nota
    return soma / len(aluno)

def resumo_turma(turma):
    for aluno in turma:
        media = calcula_media(aluno)
        print(f'notas : {aluno} | média: {media:5.2f}')

qtd_alunos = int(input('Quantidade: '))
turma = preenche_turma(qtd_alunos)
resumo_turma(turma)
```

Python Tutor

Preenchendo Matrizes e Tabelas

Para criar uma tabela usando sequências aninhadas, o procedimento é o mesmo.

Tabela com 3 registros de 4 campos
(3 linhas por 4 colunas)

	tipo	estoque	preço	importado	
produto 0	smartphone	100	1199.00	sim	← registro 0
produto 1	televisão	5	2599.00	não	← registro 1
produto 2	notebook	20	4500.00	sim	← registro 2
	↑ campo 0	↑ campo 1	↑ campo 2	↑ campo 3	

```
produtos = [['smartphone', 100, 1199.00, True ],
            ['televisão',    5, 2599.00, False],
            ['notebook',    20, 4500.00, True ]]
```

Exercícios

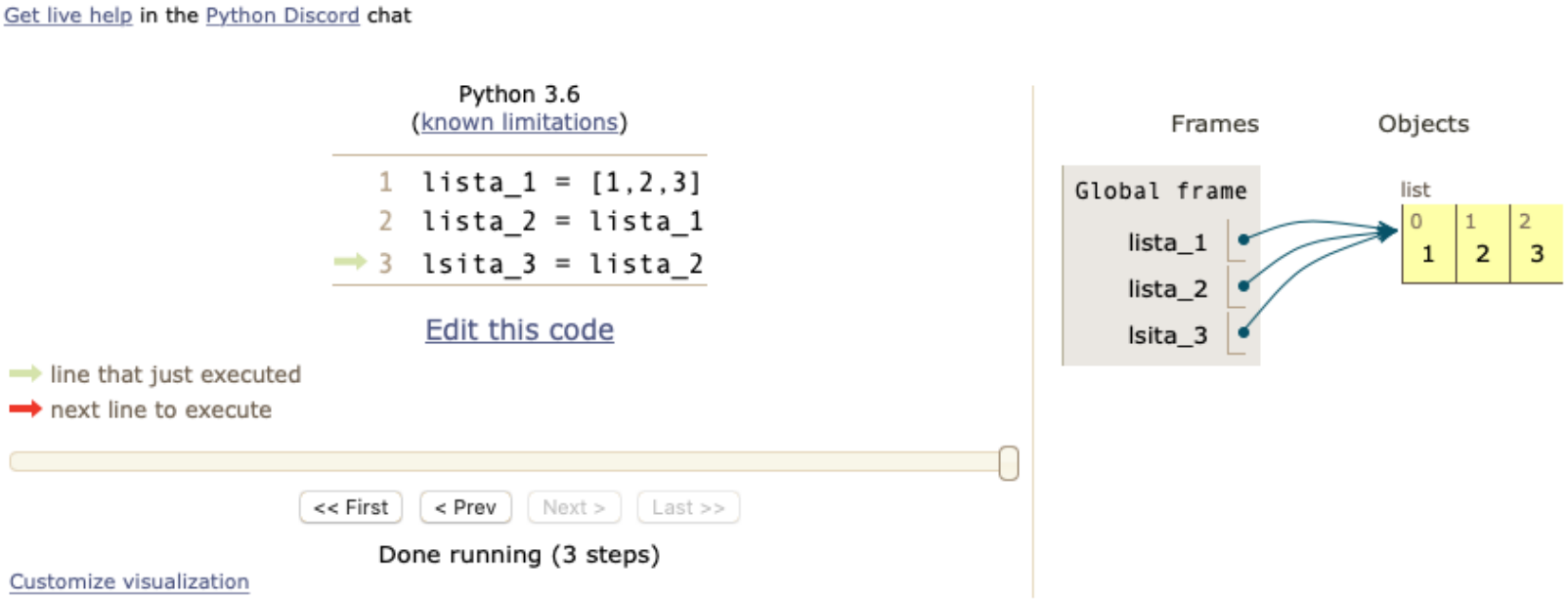
1. Crie uma função que receba uma matriz numérica como argumento e retorne o valor da soma de seus itens. Acesse os itens da matriz indexando-a com os dois índices.

2. Crie uma função que receba uma matriz numérica como argumento e retorne a matriz oposta, ou seja, uma cópia da matriz argumento com os itens com sinal invertido.

Cópia de objetos em Python

Vimos que a atribuição de um objeto a uma variável não gera uma cópia deste.

A variável receberá apenas uma referência (endereço) ao objeto que já foi criado e está na memória.

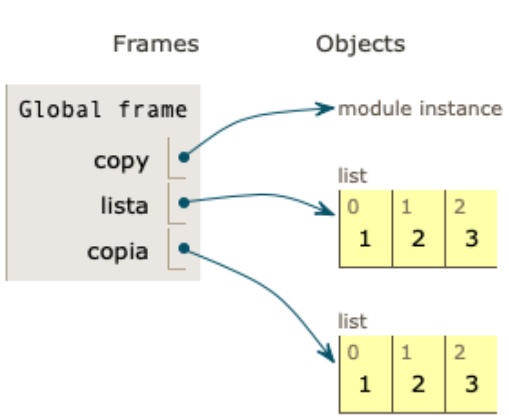


Cópia de objetos em Python

Cópia Rasa

Também chamada de cópia simples, ocorre quando apenas o primeiro nível do objeto é copiado e os demais níveis continuam referenciando os objetos originais.





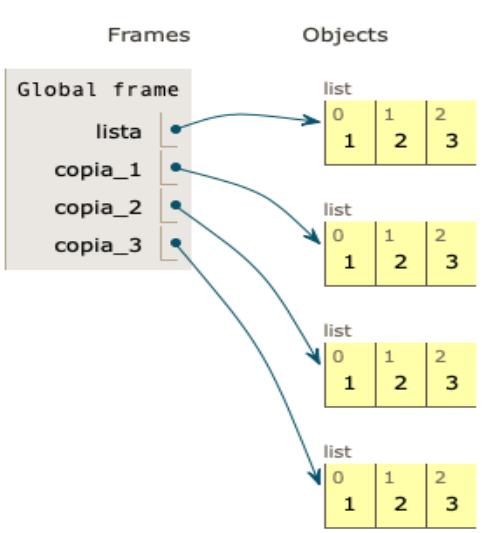
Cópia de objetos em Python

Cópia Rasa

Para uma lista s temos ainda as seguintes formas de se obter uma copia rasa:

- Fatiamento completo: s[:]
- Método copy dos objetos do tipo lista: s.copy ()
- Com a função construtora de listas: list(s)





Cópia de objetos em Python

Cópia profunda

Python 3.6
(known limitations)

```
1  lista = [[1,2,3],['a','b','c']]
2
3  copia_fake = lista
4  copia_1 = lista[:]
5  copia_2 = lista.copy()
6  copia_3 = list(lista)
7  copia_4 = copy.copy(lista)
```

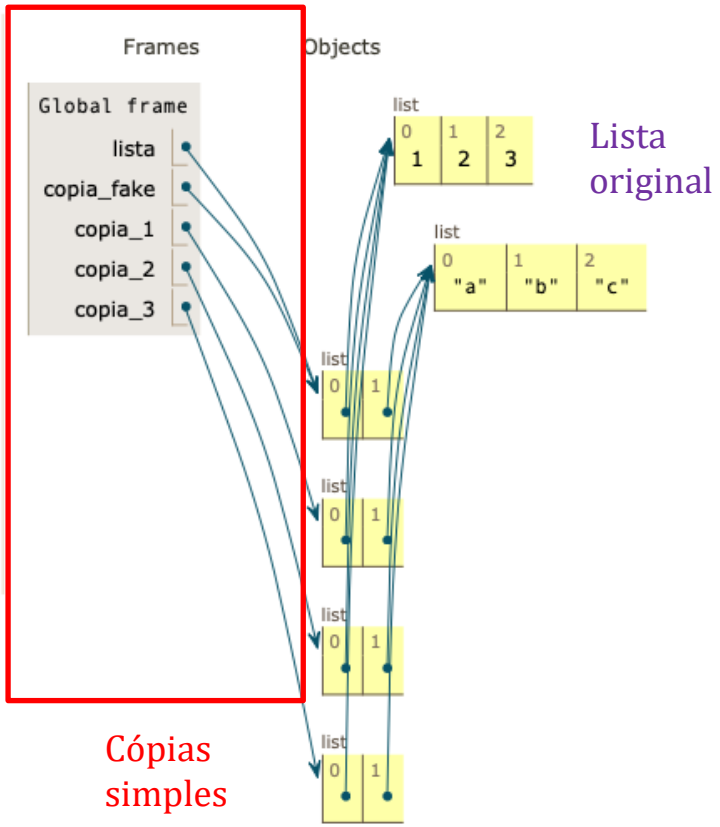
Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 6 of 6

Customize visualization



Cópia de objetos em Python

Cópias profundas

As formas que vimos para criar cópias rasas geram apenas uma cópia da lista principal, a mais externa, mas não das listas aninhadas. Em situações em que é necessário fazer cópias **totalmente desvinculadas do objeto original** é preciso criar cópias profundas, ou seja, **cópias do objeto principal e dos objetos internos a ele em todos os níveis.**

Em Python a função **deepcopy** do módulo copy faz o serviço.

Cópia de objetos em Python

Python 3.6
(known limitations)

```
1  import copy
2
3  lista = [[1,2,3], ['a','b','c']]
4  copai_fake = lista
5  copia_rasa = copy.copy(lista)
6  copia_profunda = copy.deepcopy(lista)
```

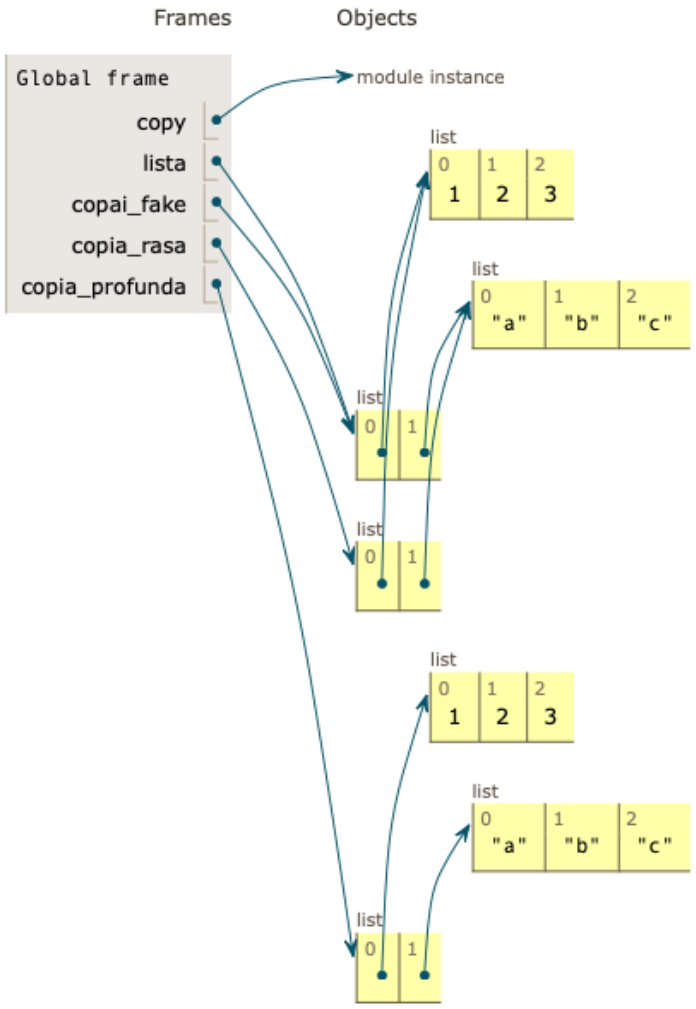
Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Done running (5 steps)

stomize visualization



Cópias de objetos imutáveis

Em Python, copiar objetos imutáveis é dispensável pois eles não podem ser alterados. No entanto, cópias profundas de objetos imutáveis que contenham objetos mutáveis como, por exemplo, tuplas de listas podem ser úteis.

Não trataremos disso!

Argumentos nomeados e com valor padrão

A função print pode ter seu comportamento alterado se estipularmos uma **string separadora de argumentos e a string terminadora de impressão**.

Não fazendo nada, o comportamento padrão coloca um espaço como separador e uma quebra de linha como terminador.

```
print('Lennon', 'McCartney', 'Harrison', 'Starr')
Lennon McCartney Harrison Starr
print('Lennon', 'McCartney', 'Harison', 'Starr', sep = ' & ', end='!')
Lennon & McCartney & Harison & Starr!
```

Argumentos nomeados

Os argumentos em Python **são posicionais**, isto é, são associados aos parâmetros de acordo com a posição em que são escritos. Esse comportamento pode ser alterado com **argumentos nomeados.**

Para isso, chama-se a função passando os argumentos como valores atribuídos **aos nomes dos parâmetros.**

Para descobrir os nomes dos parâmetros, podemos fazer as seguintes coisas:

- Consultar a documentação
- Usar o editor de código, que pode exibir a assinatura da função quando digitamos a abertura de parênteses
- Chamar a função help que recebe como argumento o nome da função que se deseja consultar.

Argumentos nomeados

Exemplo

```
def divide(numerador, denominador):
    return numerador / denominador

a = 10
b = 5
div1 = divide(numerador=a, denominador=b)
div2 = divide(denominador=b, numerador=a)
div3 = divide(numerador=b, denominador=a)
print(f'1o: {a}/{b} = {div1}')
print(f'2o: {a}/{b} = {div2}')
print(f'3o: {b}/{a} = {div3}')
```

Argumentos nomeados

Exemplo

```
def divide(numerador, denominador):
    return numerador / denominador

a = 10
b = 5
div1 = divide(numerador=a, denominador=b)
div2 = divide(denominador=b, numerador=a)
div3 = divide(numerador=b, denominador=a)
print(f'1o: {a}/{b} = {div1}')
print(f'2o: {a}/{b} = {div2}')
print(f'3o: {b}/{a} = {div3}')
```

Argumentos nomeados

Note que a ordem da passagem dos argumentos nomeados não importa,

mas é inválido passar argumentos nomeados **antes** de posicionais

```
>>>
>>> divide(6, denominador=2)
3.0
>>> divide(numerador=6, 2)
SyntaxError: positional argument follows keyword argument
>>>
```

Passar dois argumentos para o mesmo parâmetro, ou seja, um posicional e outro nomeado causará erro.

```
>>> divide(6, numerador=2)
Traceback (most recent call last):
  File "<pyshell#5>", line 1, in <module>
    divide(6, numerador=2)
TypeError: divide() got multiple values for argument 'numerador'
>>>
```

Argumentos com valor padrão

Normalmente, quando uma função é chamada, passamos a quantidade de argumentos equivalente à quantidade de parâmetros definidos em sua criação. É possível, porém criar funções com **argumentos pré-definidos.** Então, pode-se chama-las omitindo alguns desses argumentos, tornando-os opcionais**. Para criar funções com argumentos padrão basta definir parâmetros com uma atribuição de valor como no exemplo**

```
def recepcao_cliente(nome, senha=1):
    print(f'Olá {nome}, sua senha é {senha}')
```

No shell

```
recepcao_cliente('Paul')
Olá Paul, sua senha é 1
```

Argumentos com valor padrão

No shell

```
>>> recepcao_cliente('Paul',23)
Olá Paul, sua senha é 23
>>>
```

Podemos chamar a função com argumentos nomeados e a definição de argumentos padrão combinados.

```
>>> recepcao_cliente(nome='Paul')
Olá Paul, sua senha é 1
>>> recepcao_cliente(senha=23, nome= 'Paul')
Olá Paul, sua senha é 23
>>>
```

Argumentos com valor padrão

Note que: Na definição da função, parâmetro com argumento padrão não pode anteceder parâmetro sem argumento padrão.

```
def desconto(porcentagem=0.05, valor):
    return valor * porcentagem
```



Objetos mutáveis como argumentos padrão

Ao definir uma função com argumentos padrão, o Python vai criar os objetos de cada argumento apenas uma vez, no momento em que a função for definida. Logo, mesmo que a função seja chamada diversas vezes, os valores dos argumentos padrão não serão reconstruídos e eventuais alterações serão mantidas.

Crie uma função que receba como argumentos um objeto e uma lista. A função deve adicionar o objeto ao final da lista e retorná-la. Caso não seja fornecida uma lista na chamada, função deve criar uma nova lista vazia.

Objetos mutáveis como argumentos padrão

```
def adiciona_item(x, lista=[]):
    lista.append(x)
    return lista
```

```
>> x = adiciona_item(5, [1,2,3,4])
>> x
   [1, 2, 3, 4, 5]
>> y = adiciona_item('c', ['a','b'])
>> y
   ['a', 'b', 'c']
>> x = adiciona_item(10)
>> x
   [10]
>> y = adiciona_item('z')
>> y
   [10, 'z']
>> w = adiciona_item(1)
>> w
   [10, 'z', 1]
```

Objetos são adicionados sempre à mesma lista pois a cada chamada usa-se o mesmo argumento criado na definição da função

Objetos mutáveis como argumentos padrão

Para evitar a criação do objeto mutável na definição da função e gerar um **novo** a cada execução, pode-se usar o None como argumento padrão e criar o objeto necessário caso não seja passado um argumento.

```
def adiciona_item(x, lista=None):
    if lista is None:
        lista = []

    lista.append(x)
    return lista
```

```
>>> x = adiciona_item(3)
>>> x
[3]
>>> y = adiciona_item('a')
>>> y
['a']
>>> z = adiciona_item(1)
>>> z
[1]
>>> x = adiciona_item(4, x)
>>> x
[3, 4]
>>>
```

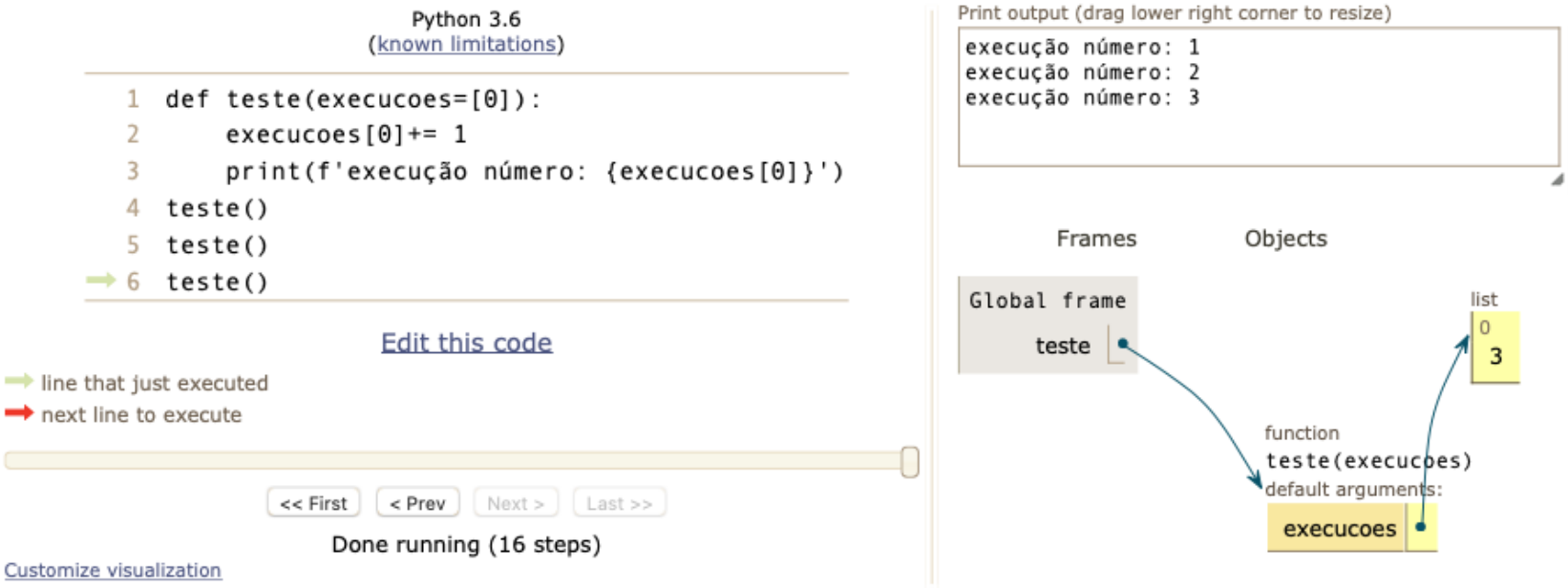
Objetos mutáveis como argumentos padrão

Há soluções que podem ser beneficiadas como a criação de uma memória temporária e compartilhada entre as chamadas de uma função, por exemplo, para implementar a **memoização** (técnica para otimizar a solução de problemas repetitivos) que não será abordada aqui.

Um exemplo de uma função com argumento padrão que é um objeto mutável pode ser vista abaixo

```
def teste(execucoes=[0]):
    execucoes[0]+= 1
    print(f'execução número: {execucoes[0]}')
```

Objetos mutáveis como argumentos padrão



O argumento do parâmetro execuções não deve ser fornecido na chamada da função pois, se for, sua execução não será contabilizada.

Python Tutor: Visualize code in Python, JavaScript, C, C++, and Java

Python 3.6
(known limitations)

```
1  def teste(execucoes=[0]):
2      execucoes[0]+= 1
3      print(f'execução número: {execucoes[0]}')
4
5  teste(execucoes=[0])
6  teste(execucoes=[0])
7  teste(execucoes=[0])
8  teste(execucoes=[0])
```

Edit this code

line that just executed
next line to execute

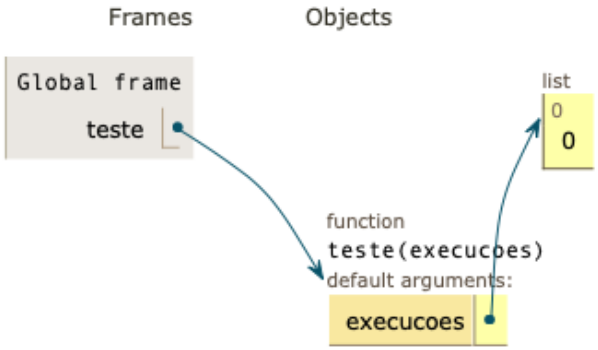
<< First | < Prev | Next > | Last >>

Done running (21 steps)

Customize visualization

Print output (drag lower right corner to resize)

```
execução número: 1
execução número: 1
execução número: 1
execução número: 1
```



FIM